

---

# Semver PostgreSQL Extension

Design Document

## Group 28

Abeeha (BSCS24010)

Maha Faisal (BSCS24048)

Musa Hassan Khan (BSCS24098)

C extension for semantic version values, comparison operators, aggregates, indexing, casts, and regression tests.

## Required topics covered

1. Extension file structure
2. PG\_MODULE\_MAGIC and PGXS
3. In-memory semver representation
4. Comparison helper design
5. SQL registration pattern
6. Regression test strategy
7. Design decision discussion
8. Detailed bonus implementation

---

## 1. Extension Structure

---

The extension is split across the standard PostgreSQL extension files. Each file has one clear responsibility, which keeps the C implementation, SQL registration, build system, and tests separate.

Makefile: uses PGXS to compile `semver.c` into the shared library `semver.so`, install `semver--1.0.sql` and `semver.control`, and register the regression test named `semver_test`.

`semver.control`: declares extension metadata such as the default version, relocatability, comment, and `module_pathname`. PostgreSQL uses this file when `CREATE EXTENSION semver` is executed.

`semver.c`: contains the C implementation. It defines the in-memory struct, input/output functions, comparison functions, utility functions, aggregate transition functions, and cast wrappers.

`semver--1.0.sql`: connects SQL names to C symbols. It creates the `semver` type, registers functions, operators, aggregates, casts, and the B-tree operator class.

`sql/semver_test.sql` and `expected/semver_test.out`: form the regression test pair used by `make installcheck`. `demo.sql` is a separate manual demo script for showing the extension behavior interactively.

## 2. Magic Block and PGXS

---

`semver.c` includes `PG_MODULE_MAGIC`. This macro places PostgreSQL module metadata inside the compiled shared library. When PostgreSQL loads `semver.so`, it checks this metadata to confirm that the library was compiled for a compatible PostgreSQL server interface.

Without the magic block, PostgreSQL may reject the shared library because it cannot safely verify the extension module layout. It is a small line of code, but it is part of the contract between a C extension and the PostgreSQL server.

The Makefile uses PGXS through `pg_config --pgxs`. PGXS supplies the PostgreSQL build rules, compiler flags, include paths, library installation paths, and regression test targets. This avoids hard-coding PostgreSQL installation directories and makes the project build like a normal PostgreSQL extension.

## 3. In-Memory Representation

---

The `semver` value is stored as a fixed-width C struct:

```
typedef struct { int32 major; int32 minor; int32 patch; } Semver;
```

The SQL type is registered with `INTERNALLENGTH = 12` and `ALIGNMENT = int4`, matching three 4-byte integer fields. A value such as 1.2.3 is stored as `major = 1`, `minor = 2`, and `patch = 3`.

The input function `semver_in` receives a `cstring` and parses the required `major.minor.patch` format. If the string is malformed, it raises an `ereport` error. The output function `semver_out` formats the three stored fields back into a display string

---

such as 1.2.3.

This representation is compact, easy to compare, and independent of the original input text. It stores the semantic data rather than a string representation.

## 4. Comparison Helper

---

All ordering logic is centralized in `semver_compare`. It compares major first, then minor, then patch. The first field that differs determines the result.

This is the most important correctness point in the extension. Semantic versions must compare numerically by component, not lexicographically as strings. For example, 1.10.0 is greater than 1.9.0 because 10 is greater than 9 in the minor field.

The six SQL comparison functions are thin wrappers over this helper: `semver_lt`, `semver_le`, `semver_eq`, `semver_ne`, `semver_ge`, and `semver_gt`. The `semver_cmp` function returns an `int4` comparison value and is used by the B-tree operator class.

## 5. SQL Registration Pattern

---

The SQL registration file follows the standard PostgreSQL C-extension pattern. First it creates a shell type with `CREATE TYPE semver`. Then it registers `semver_in` and `semver_out` as C functions. Finally it completes the type definition by assigning those functions as `INPUT` and `OUTPUT`.

After the type exists, the script registers the remaining C functions using `CREATE FUNCTION ... AS 'MODULE_PATHNAME', 'c_symbol'`. `MODULE_PATHNAME` is replaced by PostgreSQL with the installed shared library path.

The comparison operators are registered with `CREATE OPERATOR` and linked to their C functions. The aggregate functions `max(semver)` and `min(semver)` are built from `semver_larger` and `semver_smaller` transition functions.

## 6. Regression Test Strategy

---

The regression test is designed to prove both normal behavior and important edge cases. It checks basic input/output with `SELECT '1.2.3'::semver` and checks malformed input using an incomplete version such as 1.2, expecting the exact SQL error produced by `ereport`.

It exercises all six comparison operators, including `1.10.0 > 1.9.0`, which verifies that comparison is numeric and field-by-field rather than string-based. It also checks the five utility functions: `major`, `minor`, `patch`, `bump_minor`, and `is_compatible`.

Table behavior is tested by inserting `semver` values into a `packages` table, sorting with `ORDER BY`, and computing `MAX` and `MIN` aggregates. A combined query uses `is_compatible` together with `bump_minor` and `ORDER BY` to show that multiple extension features work together.

Bonus behavior is included without making the regression test overly brittle: explicit

---

casts are tested in both directions, and `CREATE INDEX USING btree` confirms that PostgreSQL accepts the `semver` type for B-tree indexing.

## 7. Design Decision: Three int32 Fields

---

A major design decision was to store semantic versions as a struct of three `int32` fields instead of packing the version into a single `int64` or storing the original string.

The struct mirrors the problem directly: semantic versions have a major, minor, and patch component. This makes parsing straightforward, makes the comparison helper readable, and avoids artificial bit-width choices that would be required for an `int64` packing scheme.

Packing into a single integer could make comparison compact, but it would require deciding how many bits each component receives and what to do when a component exceeds that range. Storing the original string would preserve formatting, but every comparison would need repeated parsing or careful string handling.

The three-field struct is therefore the clearest fit for this assignment. It is compact, fixed-width, easy to register with PostgreSQL, and directly supports the numeric comparison rules required by semantic versions.

## 8. Bonus Features

---

**Bonus 10.1: B-tree Operator Class for Indexing.** The extension registers `semver_ops` as the default B-tree operator class for the `semver` type. This lets PostgreSQL build normal B-tree indexes on `semver` columns with `CREATE INDEX packages_version_idx ON packages USING btree (version)`.

The operator class maps the `semver` comparison operators to PostgreSQL's B-tree strategy numbers: strategy 1 is `<`, strategy 2 is `<=`, strategy 3 is `=`, strategy 4 is `>=`, and strategy 5 is `>`. It also provides `semver_cmp` as support function 1, which gives PostgreSQL the three-way comparison result needed by the B-tree access method.

This bonus depends on the base comparison implementation being correct. Since `semver_cmp` uses `semver_compare`, indexed ordering follows the same numeric field-by-field rules as `ORDER BY` and the comparison operators. The `demo.sql` file includes an `EXPLAIN` query with `enable_seqscan` disabled so the output shows an Index Scan using the `semver` index.

**Bonus 10.3: Custom Casts to and from text.** The extension registers explicit casts in both directions: `text AS semver` and `semver AS text`. The `text_to_semver` wrapper parses a text value into the internal `semver` representation, while `semver_to_text` formats the stored fields back into a PostgreSQL text value.

These casts make the type easier to use with existing tables and text data. For example, users can write `some_text_column::semver` when importing version strings, and `semver_value::text` when exporting or displaying values as ordinary text. The regression test includes both `'2.3.4'::text::semver` and `'2.3.4'::semver::text`.

---

## 9. Limitations

---

The implementation supports only numeric major.minor.patch versions. It does not implement pre-release labels or build metadata.

The stored value is normalized to three integer fields, so the original input formatting is not preserved.